

Computer Vision

Lecture 6: Vision Transformers

Nguyen Manh Toan

Swinburne Vietnam

Week 6

Where We Left Off

Five weeks of architecture have all rested on one operator:

$$Z_{c,i,j} = \sum_{c'} \sum_{u,v} W_{c,c',u,v} X_{c',i+u,j+v} + b_c$$

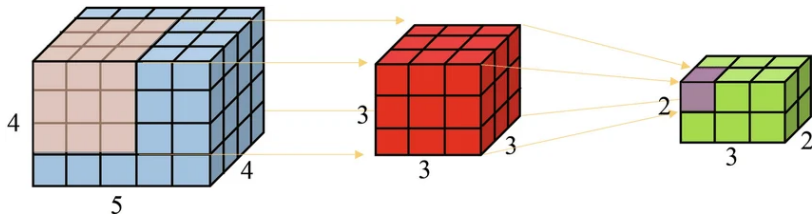


Figure: 3D convolution operation with a stride 1

- Lecture 2: local + shift-equivariant + linear $\Rightarrow$ convolution. No choice.
- Lecture 4: V1's simple and complex cells have exactly this structure

The Question To Answer

In 2020 a group at Google took a language model, fed it image patches, changed almost nothing else, and beat the best CNNs.

What that threatens

Locality and weight sharing are **assumptions**. They are correct, but a model given enough data could learn them, instead of being told.

What we will find

The assumptions are worth a great deal when data is scarce, and almost nothing when it is abundant. *Both* halves matter.

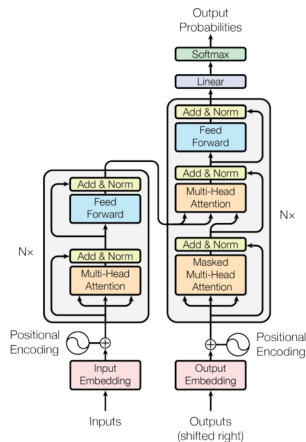


Figure: Vanilla transformer architecture

Today's Lecture

- 1 Attention
- 2 The Vision Transformer
- 3 Inductive Bias and Data
- 4 DeiT: Removing the Data Requirement
- 5 Swin: Putting the Pyramid Back
- 6 What Should You Use?

Attention

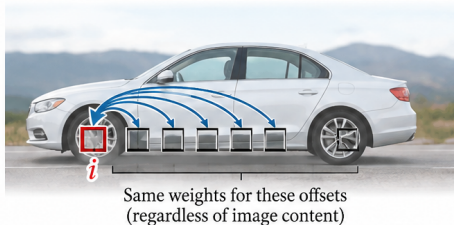
What a Convolution Cannot Do

Convolution kernel is **fixed** after training.

Consequence

How strongly pixel i influences pixel j depends only on $j - i$, never on what is actually in the image.

Convolution: fixed, offset-based weights



What we might want instead

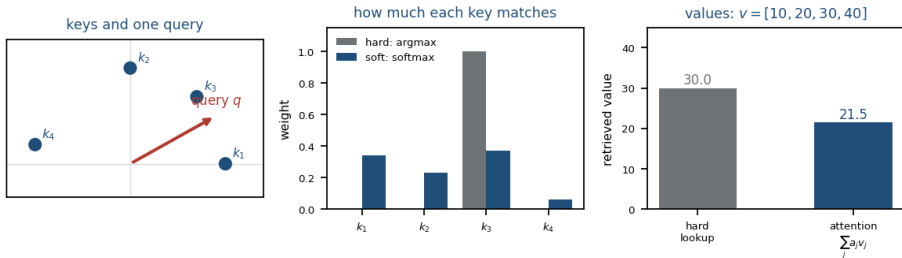
A wheel should look at the other wheel, wherever it is. The weight should depend on **content**, and be computed per image.

Attention: content-based weights (computed per image)



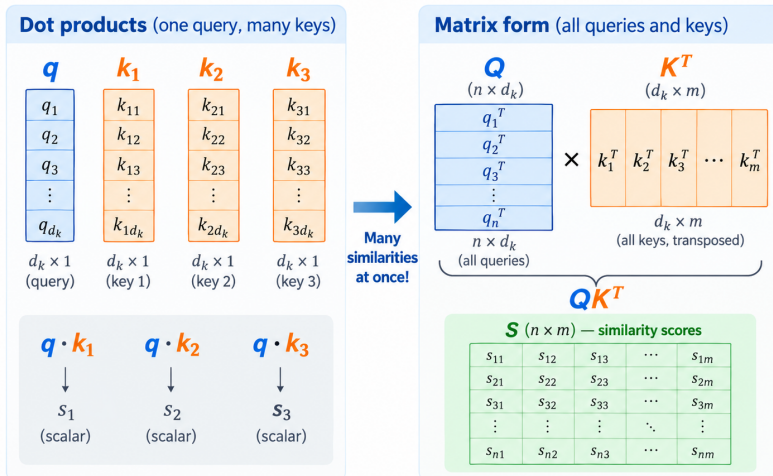
Attention Is a Soft Dictionary Lookup

- A Python dictionary: give a key, get its value, exact match only.
- Attention compares a **query** against every key, and returns a *blend* of values weighted by how well each key matched.
- The blend is **differentiable** while arg max is not.



Similarity Scores

Dot product is the cheapest similarity: one matrix multiplication.



Scaled Dot-Product Attention

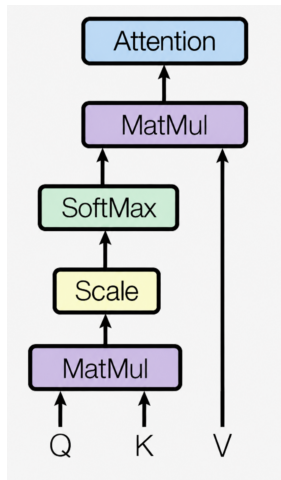
For one query $q \in \mathbb{R}^{d_k}$ against n keys and values:

$$\text{Attention}(q, K, V) = \underbrace{\text{softmax}\left(\frac{qK^\top}{\sqrt{d_k}}\right)}_{\text{weights } a \in \mathbb{R}^n, \sum_j a_j = 1} V$$

Stacking n_q queries into Q gives the form you will actually implement:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Shapes. $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{n \times d_k}$,
 $V \in \mathbb{R}^{n \times d_v}$, $QK^\top \in \mathbb{R}^{n \times n}$, **output** $\in \mathbb{R}^{n \times d_v}$

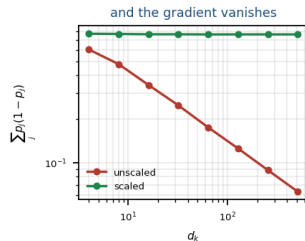
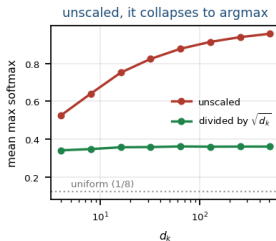
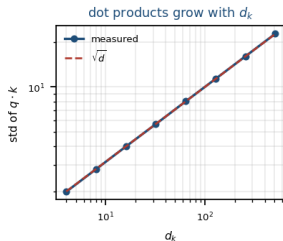


Why $\sqrt{d_k}$?

If q and k have independent, zero-mean, unit-variance components, then:

$$\text{Var}(q \cdot k) = \text{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = d_k \text{Var}(q_i) \text{Var}(k_i) = d_k \quad \implies \quad \text{std} = \sqrt{d_k}$$

Dividing by $\sqrt{d_k}$ puts the logits back at unit scale, whatever the head dimension.



Self-Attention

So far Q , K , V were given. In **self**-attention every token produces all three, from the same input $X \in \mathbb{R}^{n \times D}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

- $W_Q, W_K \in \mathbb{R}^{D \times d_k}$, $W_V \in \mathbb{R}^{D \times d_v}$ are learnable, and *these* are the parameters
- Every token asks a question (q), advertises what it has (k), and offers content (v)
- The attention weights are **not** parameters: they are recomputed for every input

Compare with CNN

A convolution's weights are learned *and* fixed.

Here the weights are *produced* by a learned function of the input — so the layer can behave differently on every image.

Number of parameters is $3D^2$ if $d_k = d_v = D$, **independent of n** .

Multi-Head Attention

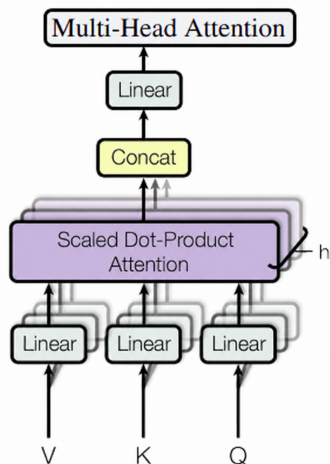
A single softmax cannot attend to two different things at once. So one attention map commits to one notion of relevance.

For h notions, run h heads in parallel, each on a $d_h = D/h$ slice:

$$\text{MSA}(X) = [\text{head}_1; \dots; \text{head}_h] W_O$$

$$\text{head}_i = \text{Attention}(XW_Q^i, XW_K^i, XW_V^i)$$

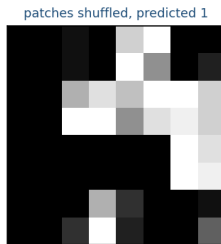
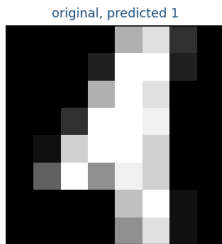
- ViT-Base: $D = 768$, $h = 12$, so $d_h = 64$. So $\sqrt{d_h} = 8$, not $\sqrt{768}$ — the scaling is per head
- Splitting rather than replicating keeps the cost **identical** to one head of width D



Attention Does Not Know Where Anything Is

Permute the input tokens and the output is permuted identically; nothing else changes.

$$\text{Attention}(PX) = P \text{ Attention}(X) \quad \text{for any permutation matrix } P$$



attention alone, tokens permuted:

$$\max | \text{Attn}(Px) - P \text{Attn}(x) | = 6.7e - 16$$

equivariant: it cannot tell
where a patch came from

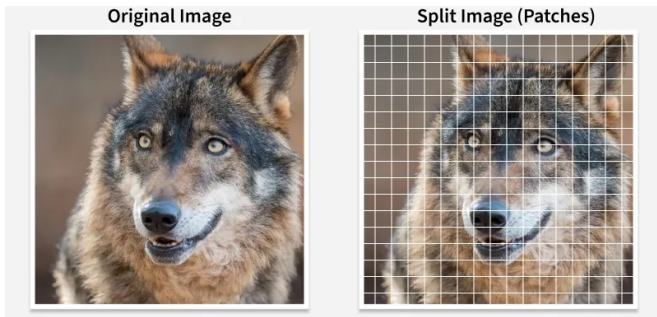
⇒ position must be **added**

A convolution *is* told where things are, by construction. Attention must be **given** position explicitly.

The Vision Transformer

An Image as a Sequence

Attention costs $O(n^2)$. One token per pixel on a 224×224 image means $n = 50,176$ and an attention matrix of 2.5 billion entries (per head, per layer).

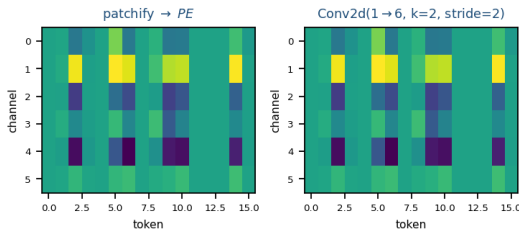


Cut the image into non-overlapping $P \times P$ patches, each is a token.

At 224×224 with $P = 16$: $n = (224/16)^2 = 196$ tokens, a sequence length a language model handles comfortably.

The Patch Embedding Is a Convolution

There are $N = HW/P^2$ patches. Each patch is flattened to P^2C numbers and projected to D dimensions by a learned $E \in \mathbb{R}^{(P^2C) \times D}$.



$\max | \cdot - \cdot | = 0.0e + 00$

the same operation,
twice

Flattening non-overlapping patches and multiplying by E is a convolution with kernel P and stride P .

So every ViT does contain one convolution. It is the layer that injects the only 2D structure the model is given.

Two Additions Before the Transformer

Two extra vectors added to image patches

The [class] token

Prepend one extra learned vector $x_{\text{class}} \in \mathbb{R}^D$ that corresponds to no patch.

- It attends to every patch in every layer
- Its final state is the image representation, fed to the classifier
- Borrowed unchanged from BERT

Sequence length becomes $N + 1$.

Position embeddings

Add a learned $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$ to the token embeddings.

- Standard **1D** embeddings — the model is not told the grid is 2D
- 2D-aware variants gave no measurable gain
- Added, not concatenated: cheaper, and empirically equivalent

An alternative to the [class] token — global average pooling over patches — works about as well; ViT chose the token for fidelity to BERT.

Why Position Embeddings?

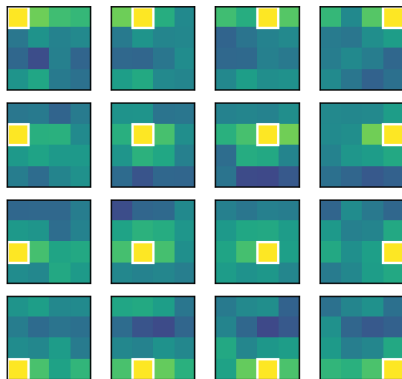
Nothing tells the model that token 5 sits below token 1. It must discover the grid.

Each tile: the cosine similarity between one patch's embedding and all sixteen, arranged in the grid. Bright near the white box = nearby patches have similar embeddings.

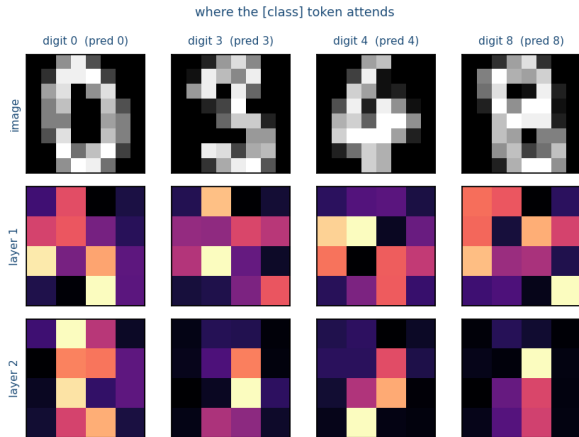
Measured on our own trained model: mean similarity 0.13 for neighbours versus -0.04 for distant patches;
 $\text{corr}(\text{distance}, \text{similarity}) = -0.38$.

This only appears if training includes **translations**. Without them every digit sits in the same place, position is just a name tag, and the structure never forms.

cosine similarity of learned position embeddings
(each tile: one patch vs. all 16; white box = itself)



What the [class] Token Looks At



The attention the classifier token pays to each patch, averaged over heads.

- The pattern is **different for every image**, the content dependence a fixed kernel cannot have
- Different between layers: the same token asks a different question at each depth
- Nothing here was supervised. No one told the model which patches matter

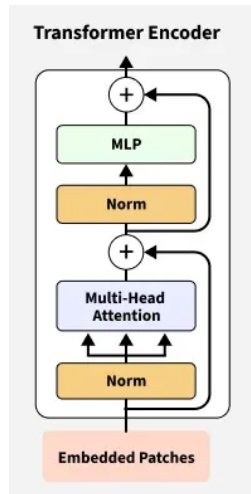
A high weight means a patch was *read*, not that it was decisive. Treat them as a hint, not an explanation.

The Transformer Block

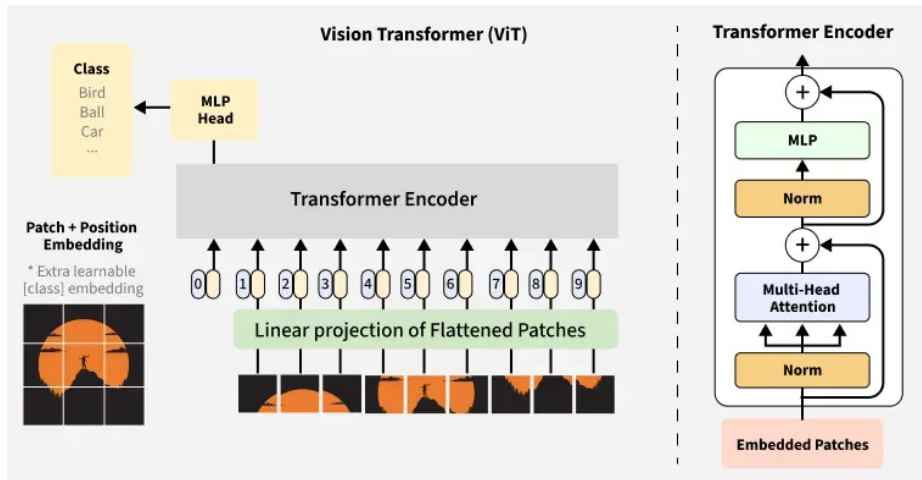
Two sublayers, each wrapped in a residual connection, each preceded by LayerNorm.

- The MLP is two layers, $D \rightarrow 4D \rightarrow D$, with **GELU**
- It acts on each token *separately* — all mixing happens in MSA
- **Pre-norm**: LN inside the residual branch, not after it

Pre-norm matters: it leaves an unobstructed identity path from input to loss, and is what lets these models train without learning-rate warmup tricks.



ViT, End to End



Model Variants

Model	Layers L	Hidden D	MLP	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Per block: $4D^2$ for attention (Q, K, V, O) plus $8D^2$ for the MLP = $12D^2$.

ViT-Base: $12 \times 12 \times 768^2 \approx 85\text{M}$ — which is the whole model.

Where the parameters go

Two thirds of a transformer's parameters are in the **MLPs**, not the attention.

Naming: ViT-B/16 means Base with 16×16 patches. *Smaller* patches mean *more* tokens and more compute — B/16 is over four times the cost of B/32.

Inductive Bias and Data

What Exactly Was Given Up

Baked into every conv layer

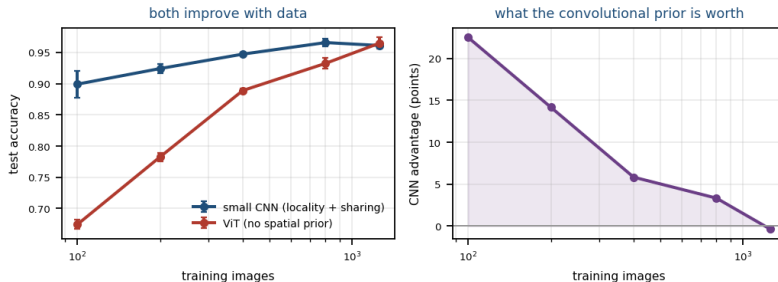
- **Locality** — a unit sees a small window
- **Translation equivariance** — shift the input, the output shifts
- **2D neighbourhood** — the grid is in the operator itself

Present in ViT

- MLP layers are per-token, hence local and equivariant
- **Self-attention is global from layer 1**
- 2D structure enters twice only: cutting patches, and interpolating position embeddings when resolution changes

A CNN is *told* how images work. A ViT must *infer* it — and inference costs data.

Measured: What the Convolutional Prior Is Worth



Our own ViT against a small CNN, same data, same budget:

- 100 images: CNN ahead by **22.5** points
- 400 images: ahead by **5.8**
- 1257 images: ahead by -0.4 — the ViT has caught up

The advantage doesn't just shrink; it **vanishes**. The prior is a substitute for data, and stops paying once the data arrives.

The Same Curve, at Google Scale

Pre-training set	Size	Outcome for ViT
ImageNet-1k	1.3M	ViT-Large <i>underperforms</i> ViT-Base; both below ResNets
ImageNet-21k	14M	ViT and ResNets comparable
JFT-300M	300M	ViT wins, and bigger ViTs win more

Best results, ImageNet top-1

- ViT-H/14 (JFT): **88.55%**
- ViT-L/16 (JFT): 87.76%
- BiT-L, ResNet152x4 (JFT): 87.54%
- ViT-L/16 (ImageNet-21k only): 85.30%

The paper's own summary

“Large scale training trumps inductive bias.”

Also reported: some attention heads span most of the image already in layer 1, while others stay local — the network rediscovers locality where it is useful, rather than being given it. *16-token model is far too small to show this; every head sits at the uniform baseline.*

DeiT: Removing the Data Requirement

ViT's result came with a footnote: **300 million proprietary images** and a TPU cluster.

DeiT's claim (Touvron et al., 2021)

The architecture was never the problem.
DeiT-B is byte-for-byte identical to ViT-B — only the training recipe changed.

The result

- ImageNet-1k only, no external data
- 86M parameters, **81.8%** top-1
- One machine, under three days

If the architecture is unchanged and the data is $230\times$ smaller, everything that follows is about **regularisation and augmentation**.

The Recipe Is the Contribution

Augmentation (all from Lecture 5)

- RandAugment
- Mixup and CutMix
- Random erasing
- Repeated augmentation

Regularisation and optimisation

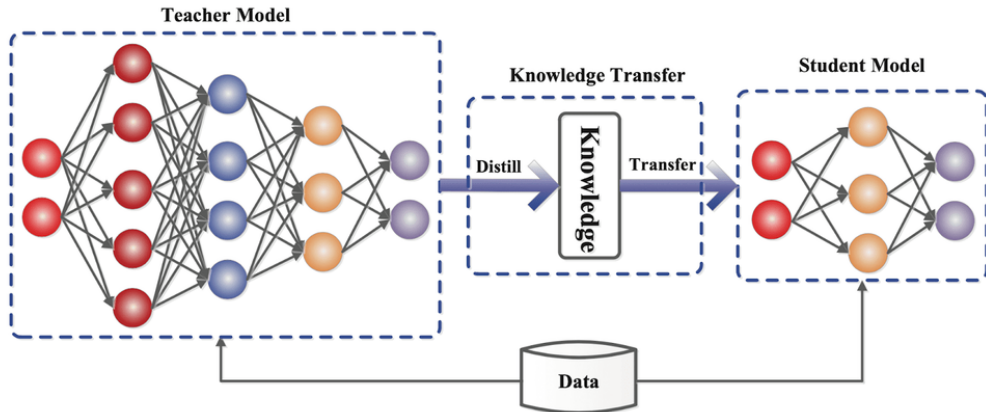
- Stochastic depth
- Label smoothing
- AdamW with cosine decay
- Careful warmup

The lesson

A “transformers need 300M images” result was really a “this model was undertrained” result.
Before concluding an architecture is wrong for your data, exhaust the recipe.

Knowledge Distillation

Standard distillation trains a student to match a teacher's soft outputs



Knowledge Distillation

Train a **student** to reproduce a **teacher**'s output distribution instead of — or alongside — the true label (Hinton et al., 2015):

$$\mathcal{L} = (1 - \lambda) \underbrace{\text{CE}(p_s, y)}_{\text{the real label}} + \lambda T^2 \underbrace{\text{KL}(p_t^T \| p_s^T)}_{\text{agree with the teacher}}, \quad p_i^T = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Why the teacher helps at all

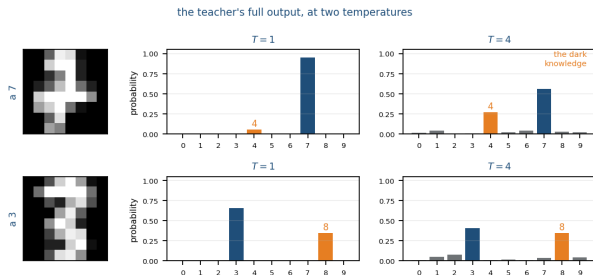
A one-hot label says “this is a 7” and nothing else. The teacher also says *what else it nearly was* — and those relative probabilities encode which classes look alike. Hinton called it **dark knowledge**.

What T does

At $T = 1$ a confident teacher is nearly one-hot and says little. Raising T flattens the distribution and exposes the runner-ups.

The T^2 factor restores the gradient magnitude, which softmax otherwise shrinks by $1/T^2$.

Dark Knowledge



Our small CNN (97.4% accurate) on two test digits. On a **7** its runner-up is **4**; on a **3** it is **8**.

Neither is random — those are the digits a human confuses too. The teacher has learned a similarity structure over classes that the labels never stated.

Target	Entropy (nats)
Hard label	0
Teacher, $T = 1$	0.135
Teacher, $T = 4$	1.343
Uniform	2.303

A hard label carries nothing beyond the answer.

The Distillation Token

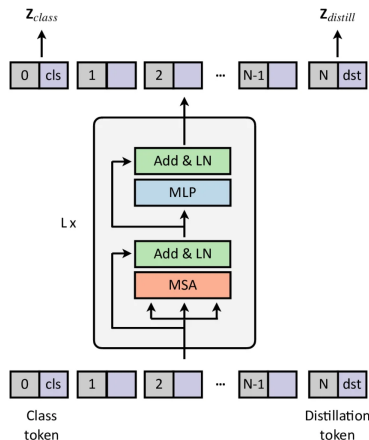
DeiT adds the teacher *inside the sequence*:

- A third token type, alongside [class] and the patches
- It attends and is attended to like any other token
- Its target is the **teacher's prediction**; [class] keeps the true label
- At test time the two predictions are averaged

The two tokens converge to genuinely different solutions — they agree far less often than two [class] tokens would.

Results, ImageNet top-1

DeiT-B	81.8%
DeiT-B, distilled	83.4%
DeiT-B $\uparrow$ 384	83.1%
DeiT-B distilled $\uparrow$ 384	85.2%



The Finding

A transformer student learns **more from a convolutional teacher** than from a transformer teacher of equal accuracy.

Teacher	Teacher acc.	DeiT-B student, distilled
DeiT-B (transformer)	81.8	81.9
RegNetY-4GF (convnet)	80.0	82.7
RegNetY-16GF (convnet)	82.9	83.0

Why this is interesting

A *weaker* convnet teacher (80.0) beats a *stronger* transformer teacher (81.8). The student is not copying accuracy — it is inheriting the convolutional **inductive bias**, transmitted through the teacher's outputs.

Which closes the loop: the bias can be handed over as data, if you cannot build it in.

Swin: Putting the Pyramid Back

What ViT Cannot Do

1. One scale

Every ViT layer works at $16\times$ downsampling. But detection needs to find objects at many sizes, and segmentation needs per-pixel output.

Every detector since Lecture 5 assumed a **feature pyramid**. ViT has none.

2. Quadratic cost

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C$$

The second term is quadratic in the number of patches. Fine for $n = 196$; hopeless for the high-resolution inputs dense prediction needs.

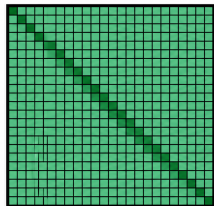
Swin (Liu et al., ICCV 2021 best paper) fixes both, and the fix in each case is a concept from Lecture 2.

Window Attention: the Complexity Argument

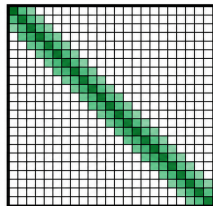
Restrict attention to non-overlapping $M \times M$ windows:

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C \quad \longrightarrow \quad \Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$$

With M **fixed** at 7, the second term becomes linear in hw .



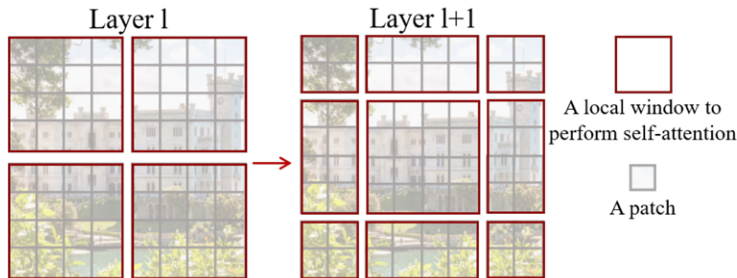
Full n^2 attention



Sliding window attention

Shifted Windows

Fixed windows never exchange information: a patch on one side of a boundary can never influence its neighbour on the other. So alternate the partition between layers.

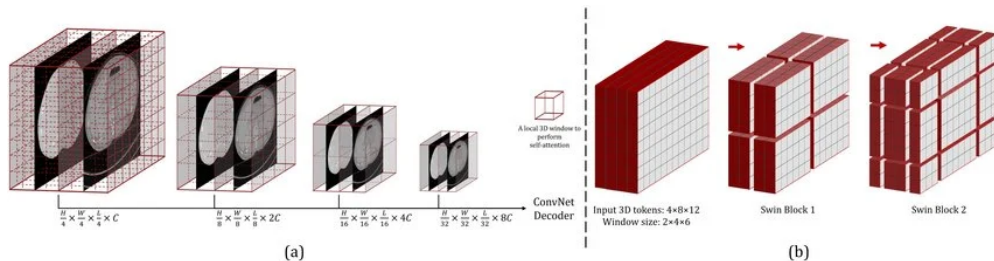


$$\hat{z}^\ell = \text{W-MSA}(\text{LN}(z^{\ell-1})) + z^{\ell-1}$$
$$\hat{z}^{\ell+1} = \text{SW-MSA}(\text{LN}(z^\ell)) + z^\ell$$

Shifting creates more, smaller windows at the edges. Swin avoids the waste with a **cyclic shift** plus a mask, keeping the window count — and the cost — unchanged.

Blocks come in pairs: regular, then shifted by $\lfloor M/2 \rfloor$.

Patch Merging Rebuilds the Hierarchy



Between stages: concatenate each 2×2 group of neighbouring tokens ($4C$ features), then project to $2C$.

- Tokens $\div 4$, channels $\times 2$
- Stages at $4 \times, 8 \times, 16 \times, 32 \times$

Recognition

Halve the resolution, double the channels (VGG's rule). Swin is a transformer wearing a CNN's skeleton.

Swin Results

Task	Swin	Previous best
ImageNet-1k top-1	87.3%	—
COCO test-dev box AP	58.7	+2.7 improvement
COCO test-dev mask AP	51.1	+2.6 improvement
ADE20K val mIoU	53.5	+3.2 improvement

Why the dense-prediction gains are the big ones

Classification was already close. Detection and segmentation are where the pyramid and the linear cost actually matter — and those margins are large.

What Swin really demonstrates

The winning recipe was not “attention instead of convolution” but **attention plus the structural priors convolution had all along**.

What Should You Use?

What attention genuinely brings

- Content-dependent weights, recomputed per image
- Global interaction from the first layer
- Scales better than CNNs when data is enormous
- One architecture shared with language $\Rightarrow$ multimodal models

What the convolutional priors still buy

- Far better accuracy per training image
- Linear cost in image size
- Multi-scale features for free
- Swin and DeiT both *re-import* them

ConvNeXt is the control experiment (Lecture 5)

Give a ResNet the transformer *training recipe* and design choices, keep convolution, and it matches Swin. Much of the 2020–21 gap was recipe, not operator.

Situation	Reach for
A few thousand images, training from scratch	A small CNN. Not close.
Any size, fine-tuning a pretrained model	Either — pretraining erases the gap
Detection or segmentation	Swin, or a ConvNeXt/ResNet backbone
Text and images together	A transformer, for the shared interface
Tight compute or latency budget	MobileNet / EfficientNet

The practical rule

Almost nobody trains a ViT from scratch. **Fine-tune a pretrained one** — which is how the 300M images end up working for you too.

Summary

- ① Attention is a differentiable soft dictionary lookup; the weights depend on **content**, not offset
- ② $\sqrt{d_k}$ is the Lecture 5 variance argument, protecting softmax from saturation
- ③ Self-attention is permutation-equivariant, so position must be added explicitly
- ④ ViT = patches + [class] token + position + a stack of pre-norm blocks. Four equations
- ⑤ Dropping the convolutional priors costs data — we measured 22.5 points at 100 images, and zero at 1257
- ⑥ DeiT: the architecture was fine, the recipe was not
- ⑦ Swin: windows and a pyramid — locality and multi-scale, deliberately restored
- ⑧ For your project: fine-tune. Train from scratch only with a CNN

- Dosovitskiy et al., “An Image is Worth 16×16 Words”, *ICLR* 2021 — the ViT paper
- Vaswani et al., “Attention Is All You Need”, *NeurIPS* 2017 — where scaled dot-product attention comes from
- Touvron et al., “Training data-efficient image transformers”, *ICML* 2021 — DeiT
- Liu et al., “Swin Transformer”, *ICCV* 2021 — shifted windows, best paper
- Liu et al., “A ConvNet for the 2020s”, *CVPR* 2022 — ConvNeXt, the rebuttal

Code worth reading: `timm (vision_transformer.py)` is about 300 readable lines and matches the four equations almost exactly.

lecture06_handson.ipynb: write attention from scratch in NumPy.

- 1 Scaled dot-product attention, and the $\sqrt{d_k}$ experiment
- 2 Multi-head attention, gradient-checked
- 3 Patch embedding = strided convolution
- 4 Permutation equivariance, demonstrated
- 5 A complete transformer block
- 6 Train a real ViT on digits and read its attention maps
- 7 The data-efficiency crossover, reproduced
- 8 The same model in PyTorch